

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “ Парадигмы и конструкции языков программирования”

Отчет по ЛР № 6
“Модульное тестирование”

A handwritten signature in blue ink, enclosed in a hand-drawn oval. The signature appears to be 'Kam'.

Выполнил:
Студент группы ИУ5-36Б
Калмыков А.Ю.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Листинг кода

equation_solver.py

```
import math
from typing import List, Tuple

class EquationSolver:
    @staticmethod
    def solve_biquadratic(a: float, b: float, c:
float) -> List[float]:
        if abs(a) < 1e-10:
            raise ValueError("Коэффициент 'a' не
может быть равен нулю")

        discriminant = b**2 - 4*a*c

        roots = []

        if discriminant < -1e-10:
            return []

        if abs(discriminant) < 1e-10:
            y = -b / (2*a)
            roots.extend(EquationSolver._get_roots_fr
om_y(y))

        else:
            sqrt_d = math.sqrt(discriminant)
            y1 = (-b + sqrt_d) / (2*a)
            y2 = (-b - sqrt_d) / (2*a)

            roots.extend(EquationSolver._get_roots_fr
om_y(y1))
```

```
        roots.extend(EquationSolver._get_roots_from_y(y2))
```

```
    unique_roots =  
EquationSolver._remove_duplicates(sorted(roots))
```

```
    return unique_roots
```

```
@staticmethod
```

```
def _get_roots_from_y(y: float) -> List[float]:
```

```
    roots = []
```

```
    if abs(y) < 1e-10:
```

```
        roots.append(0.0)
```

```
    elif y > 0:
```

```
        sqrt_y = math.sqrt(y)
```

```
        roots.append(sqrt_y)
```

```
        roots.append(-sqrt_y)
```

```
    return roots
```

```
@staticmethod
```

```
def _remove_duplicates(numbers: List[float],  
tolerance: float = 1e-10) -> List[float]:
```

```
    if not numbers:
```

```
        return []
```

```
    unique = [numbers[0]]
```

```
    for num in numbers[1:]:
```

```
        if abs(num - unique[-1]) > tolerance:
```

```
            unique.append(num)
```

```
    return unique
```

```

    @staticmethod
    def validate_coefficients(a: float, b: float, c:
float) -> Tuple[bool, str]:
        if math.isnan(a) or math.isnan(b) or
math.isnan(c):
            return False, "Коэффициенты не могут быть
NaN"

        if math.isinf(a) or math.isinf(b) or
math.isinf(c):
            return False, "Коэффициенты не могут быть
бесконечными"

        if abs(a) < 1e-10:
            return False, "Коэффициент 'a' не может
быть равен нулю"

        return True, ""

```

test_equation_tdd.py

```

import unittest
import math
from equation_solver import EquationSolver

class TestEquationSolverTDD(unittest.TestCase):
    """TDD тесты для решения биквадратного
уравнения"""

    def test_no_real_roots(self):
        """Тест: уравнение не имеет действительных
корней"""
        # Уравнение:  $x^4 + 2x^2 + 5 = 0$ 
        roots = EquationSolver.solve_biquadratic(1,
2, 5)

```

```

self.assertEqual(len(roots), 0)

def test_four_real_roots(self):
    """Тест: уравнение имеет 4 действительных
    корня"""
    # Уравнение:  $x^4 - 5x^2 + 4 = 0$ 
    # Корни: -2, -1, 1, 2
    roots = EquationSolver.solve_biquadratic(1, -
5, 4)
    self.assertEqual(len(roots), 4)

    expected_roots = [-2, -1, 1, 2]
    for expected, actual in zip(expected_roots,
roots):
        self.assertAlmostEqual(expected, actual,
places=10)

def test_one_root_zero(self):
    """Тест: уравнение имеет один корень  $x=0$ """
    # Уравнение:  $x^4 + 2x^2 = 0$ 
    roots = EquationSolver.solve_biquadratic(1,
2, 0)
    self.assertEqual(len(roots), 1)
    self.assertAlmostEqual(roots[0], 0.0,
places=10)

def test_two_real_roots(self):
    """Тест: уравнение имеет 2 действительных
    корня"""
    # Уравнение:  $x^4 - 4x^2 = 0$ 
    # Корни: -2, 2
    roots = EquationSolver.solve_biquadratic(1, -
4, 0)
    self.assertEqual(len(roots), 2)

```

```

        self.assertAlmostEqual(roots[0], -2.0,
places=10)
        self.assertAlmostEqual(roots[1], 2.0,
places=10)

    def test_three_real_roots(self):
        """Тест: уравнение имеет 3 действительных
корня"""
        # Уравнение:  $x^4 - x^2 = 0$ 
        # Корни: -1, 0, 1
        roots = EquationSolver.solve_biquadratic(1, -
1, 0)
        self.assertEqual(len(roots), 3)
        expected_roots = [-1, 0, 1]
        for expected, actual in zip(expected_roots,
roots):
            self.assertAlmostEqual(expected, actual,
places=10)

    def test_coefficient_a_zero_raises_error(self):
        """Тест: коэффициент  $a=0$  вызывает
исключение"""
        with self.assertRaises(ValueError) as
context:
            EquationSolver.solve_biquadratic(0, 2, 3)
            self.assertIn("не может быть равен нулю",
str(context.exception))

    def test_double_root(self):
        """Тест: уравнение с кратными корнями"""
        # Уравнение:  $x^4 - 2x^2 + 1 = 0$ 
        # Корни: -1, 1 (каждый корень кратности 2)
        roots = EquationSolver.solve_biquadratic(1, -
2, 1)
        self.assertEqual(len(roots), 2)

```

```

        self.assertAlmostEqual(roots[0], -1.0,
places=10)

        self.assertAlmostEqual(roots[1], 1.0,
places=10)

    def test_validation_valid_coefficients(self):
        """Тест: валидация корректных
коэффициентов"""
        valid, message =
EquationSolver.validate_coefficients(1, 2, 3)
        self.assertTrue(valid)
        self.assertEqual(message, "")

    def test_validation_invalid_coefficients(self):
        """Тест: валидация некорректных
коэффициентов"""
        # a = 0
        valid, message =
EquationSolver.validate_coefficients(0, 2, 3)
        self.assertFalse(valid)
        self.assertIn("не может быть равен нулю",
message)

        # NaN
        valid, message =
EquationSolver.validate_coefficients(float('nan'), 2,
3)
        self.assertFalse(valid)
        self.assertIn("NaN", message)

if __name__ == '__main__':
    unittest.main()

```

equation_steps.py

```

from behave import given, when, then, step
import math
from equation_solver import EquationSolver

@given('биквадратное уравнение с коэффициентами
      a={a}, b={b}, c={c}')
def step_given_equation_with_coefficients(context, a,
                                          b, c):
    """Задаёт коэффициенты уравнения"""
    context.a = float(a)
    context.b = float(b)
    context.c = float(c)

    @when('я решаю уравнение')
    def step_when_solve_equation(context):
        """Решает уравнение и сохраняет результат"""
        try:
            context.roots =
EquationSolver.solve_biquadratic(
            context.a, context.b, context.c
        )
            context.error = None
        except Exception as e:
            context.error = str(e)
            context.roots = []

    @when('я пытаюсь решить уравнение')
    def step_when_try_solve_equation(context):
        """Пытается решить уравнение (ожидается возможная
        ошибка)"""
        step_when_solve_equation(context)

    @then('я получаю {expected_count} действительных
          корней')

```



```

def step_then_get_roots_count(context,
    expected_count):
    """Проверяет количество корней"""
    expected = int(expected_count)
    actual = len(context.roots)
    assert actual == expected, (
        f"Ожидалось {expected} корней, но получено {actual}. "
        f"Корни: {context.roots}"
    )

    @then('я получаю ошибку "{expected_error}"')
def step_then_get_error(context, expected_error):
    """Проверяет, что получена ожидаемая ошибка"""
    assert context.error is not None, "Ошибка не была вызвана"

    assert expected_error in context.error, (
        f"Ожидалась ошибка содержащая '{expected_error}', "
        f"но получено: '{context.error}'"
    )

    @then('корни равны {expected_roots} с точностью {tolerance}')
def step_then_roots_equal(context, expected_roots,
    tolerance):
    """Проверяет значения корней с заданной точностью"""

    # Преобразуем строку с корнями в список чисел
    import ast
    expected = ast.literal_eval(expected_roots)
    tol = float(tolerance)

    assert len(context.roots) == len(expected), (

```

```

        f"Количество корней не совпадает: "
        f"ожидалось {len(expected)}, получено "
        f"{len(context.roots)}"
    )

    # Сортируем корни для сравнения
    actual_sorted = sorted(context.roots)
    expected_sorted = sorted(expected)

    for i, (actual, expected_val) in
        enumerate(zip(actual_sorted, expected_sorted)):
        assert math.isclose(actual, expected_val,
            abs_tol=tol), (
            f"Корень #{i+1}: ожидалось "
            f"{expected_val}, "
            f"получено {actual}, разница {abs(actual
            - expected_val)}"
        )

@then('корень равен {expected_value} с точностью
      {tolerance}')
def step_then_root_equals(context, expected_value,
    tolerance):
    """Проверяет значение единственного корня"""
    expected = float(expected_value)
    tol = float(tolerance)

    assert len(context.roots) == 1, f"Ожидался 1
        корень, получено {len(context.roots)}"
    assert math.isclose(context.roots[0], expected,
        abs_tol=tol), (
        f"Корень: ожидался {expected}, "
        f"получен {context.roots[0]}"
    )

```

equation.feature

language: ru

Функция: Решение биквадратного уравнения

Биквадратное уравнение имеет вид: $a*x^4 + b*x^2 + c = 0$

Чтобы найти корни уравнения

Как пользователь

Я хочу вводить коэффициенты и получать правильные корни

Сценарий: Уравнение не имеет действительных корней

Дано биквадратное уравнение с коэффициентами $a=1$, $b=2$, $c=5$

Когда я решаю уравнение

Тогда я получаю 0 действительных корней

Сценарий: Уравнение имеет 4 действительных корня

Дано биквадратное уравнение с коэффициентами $a=1$, $b=-5$, $c=4$

Когда я решаю уравнение

Тогда я получаю 4 действительных корня

И корни равны $[-2, -1, 1, 2]$ с точностью 0.0001

Сценарий: Уравнение имеет один корень ($x=0$)

Дано биквадратное уравнение с коэффициентами $a=1$, $b=2$, $c=0$

Когда я решаю уравнение

Тогда я получаю 1 действительный корень

И корень равен 0 с точностью 0.0001

Сценарий: Уравнение имеет 2 действительных корня

Дано биквадратное уравнение с коэффициентами $a=1$, $b=-4$, $c=0$

Когда я решаю уравнение

Тогда я получаю 2 действительных корня

И корни равны $[-2, 2]$ с точностью 0.0001

Сценарий: Коэффициент a равен нулю

Дано биквадратное уравнение с коэффициентами $a=0$,
 $b=2$, $c=3$

Когда я пытаюсь решить уравнение

Тогда я получаю ошибку "Коэффициент ' a ' не может
быть равен нулю"

Сценарий: Уравнение имеет 3 действительных корня

Дано биквадратное уравнение с коэффициентами $a=1$,
 $b=-1$, $c=0$

Когда я решаю уравнение

Тогда я получаю 3 действительных корня

И корни равны $[-1, 0, 1]$ с точностью 0.0001

run_tests.py

```
#!/usr/bin/env python3
```

```
"""
```

Утилита для запуска всех тестов

```
"""
```

```
import subprocess
```

```
import sys
```

```
import os
```

```
def run_unittests():
```

```
    """Запуск TDD тестов (unittest)"""
```

```
    print("=" * 60)
```

```
    print("Запуск TDD тестов (unittest)")
```

```
    print("=" * 60)
```

```

import unittest

from test_equation_tdd import
TestEquationSolverTDD


# Создаем test suite

suite =
unittest.TestLoader().loadTestsFromTestCase(TestEquat
ionSolverTDD)


# Запускаем тесты

runner = unittest.TextTestRunner(verbosity=2)
result = runner.run(suite)


return result.wasSuccessful()


def run_behave_tests():
    """Запуск BDD тестов (behave)"""
    print("\n" + "=" * 60)
    print("Запуск BDD тестов (behave)")
    print("=" * 60)

    try:
        # Запускаем behave через subprocess
        result = subprocess.run(
            ['behave', 'features/equation.feature',
 '--no-capture'],
            capture_output=True,
            text=True
        )

        print(result.stdout)
        if result.stderr:

```

```

        print("Ошибки:", result.stderr)

    return result.returncode == 0
except FileNotFoundError:
    print("Ошибка: behave не установлен.
Установите: pip install behave")
    return False

def main():
    """Основная функция запуска тестов"""
    print("Тестирование решения биквадратного
уравнения")
    print("=" * 60)

    # Проверяем наличие необходимых модулей
    try:
        import behave
    except ImportError:
        print("Внимание: behave не установлен. BDD
тесты пропущены.")
        print("Установите: pip install behave")
        behave_available = False
    else:
        behave_available = True

    # Запускаем TDD тесты
    tdd_success = run_unittests()

    # Запускаем BDD тесты если установлен behave
    bdd_success = True
    if behave_available:
        bdd_success = run_behave_tests()

```

```
# Итоговый результат
print("\n" + "=" * 60)
print("ИТОГИ:")
print("=" * 60)

print(f"TDD тесты: {'ПРОЙДЕНЫ' if tdd_success
else 'НЕ ПРОЙДЕНЫ'}")

if behave_available:
    print(f"BDD тесты: {'ПРОЙДЕНЫ' if bdd_success
else 'НЕ ПРОЙДЕНЫ'}")

overall_success = tdd_success and (not
behave_available or bdd_success)

print(f"\nОбщий результат: {'ВСЕ ТЕСТЫ ПРОЙДЕНЫ'
if overall_success else 'ТЕСТЫ НЕ ПРОЙДЕНЫ'}")

sys.exit(0 if overall_success else 1)

if __name__ == '__main__':
    main()
```

Скриншоты работы программы

```
PS C:\Users\vrema\Documents\GitHub\IU5_3sem_PiKYAP\Lab_4_5> & C:/Users/vreme/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vreme/Documents/GitHub/IU5_3sem_PiKYAP/Lab_6/run_tests.py
Тестирование решения биквадратного уравнения
=====
Запуск TDD тестов (unittest)
=====
test_coefficient_a_zero_raises_error (test_equation_tdd.TestEquationSolverTDD.test_coefficient_a_zero_raises_error)
Тест: коэффициент a=0 вызывает исключение ... ok
test_double_root (test_equation_tdd.TestEquationSolverTDD.test_double_root)
Тест: уравнение с кратными корнями ... ok
test_four_real_roots (test_equation_tdd.TestEquationSolverTDD.test_four_real_roots)
Тест: уравнение имеет 4 действительных корня ... ok
test_no_real_roots (test_equation_tdd.TestEquationSolverTDD.test_no_real_roots)
Тест: уравнение не имеет действительных корней ... ok
test_one_root_zero (test_equation_tdd.TestEquationSolverTDD.test_one_root_zero)
Тест: уравнение имеет один корень x=0 ... ok
test_three_real_roots (test_equation_tdd.TestEquationSolverTDD.test_three_real_roots)
Тест: уравнение имеет 3 действительных корня ... ok
test_two_real_roots (test_equation_tdd.TestEquationSolverTDD.test_two_real_roots)
Тест: уравнение имеет 2 действительных корня ... FAIL
test_validation_invalid_coefficients (test_equation_tdd.TestEquationSolverTDD.test_validation_invalid_coefficients)
Тест: валидация некорректных коэффициентов ... ok
test_validation_valid_coefficients (test_equation_tdd.TestEquationSolverTDD.test_validation_valid_coefficients)
Тест: валидация корректных коэффициентов ... ok
=====
FAIL: test_two_real_roots (test_equation_tdd.TestEquationSolverTDD.test_two_real_roots)
Тест: уравнение имеет 2 действительных корня
-----
Traceback (most recent call last):
  File "c:\Users\vrema\Documents\GitHub\IU5_3sem_PiKYAP\Lab_6\test_equation_tdd.py", line 37, in test_two_real_roots
    self.assertEqual(len(roots), 2)
    ~~~~~^~~~~~
AssertionError: 3 != 2
-----
Ran 9 tests in 0.004s

FAILED (failures=1)
```